Music Playlist Manager – Sprint 2 Backlog

Software Architecture Design 2026

Corso di Laurea Magistrale in Ingegneria Informatica
Anno Accademico 2025/2026



Docenti: Ritrovato Pierluigi, Apicella Andrea

GitHub: `https://github.com/fra343/Gruppo2-SAD`

Trello: `https://trello.com/b/Mpb9bjYE/sad`

Gruppo 02:

Apicella Francesco	IE22700106
Campagna Lorenzo	IE22700105
Cutolo Raffaele	IE22700113
Gioia Errico	IE22700112

Contents

1 Introduzione **2**
1.1 Obiettivo dello Sprint . . . 2
1.2 Stima della velocity . . . 2
1.3 User Stories selezionate . . . 2

2 Task Breakdown e assegnazioni **3**
2.1 Legenda e scala di stima . . . 3
2.2 US 0 – Setup condiviso del motore di riproduzione (10 ore-uomo totali) . . . 3
2.3 Lorenzo Campagna (LC): US-07a + US-07b (5 SP) . . . 4
2.4 Francesco Apicella (FA): US-08 + US-09b (6 SP) . . . 4
2.5 Raffaele Cutolo (RC): US-10 + US-11 (6 SP) . . . 5
2.6 Errico Gioia (EG): US-12 + US-09a (7 SP) . . . 6

3 Assegnazione delle code review **6**

4 Dettaglio delle User Stories **6**
4.1 US-07a: Avvio e pausa della riproduzione *(Assegnato a: Lorenzo Campagna)* . . . 6
4.2 US-07b: Skip della riproduzione *(Assegnato a: Lorenzo Campagna)* . . . 7
4.3 US-08: Scelta della modalità di riproduzione *(Assegnato a: Francesco Apicella)* . . . 7
4.4 US-09b: Rimozione di una traccia durante la riproduzione *(Assegnato a: Francesco Apicella)* . . . 7
4.5 US-10: Pannello “Now Playing” in tempo reale *(Assegnato a: Raffaele Cutolo)* . . . 7
4.6 US-11: Evidenziazione della traccia corrente *(Assegnato a: Raffaele Cutolo)* [US buffer] 8
4.7 US-12: Annullamento di un’operazione, undo *(Assegnato a: Errico Gioia)* [US buffer] . 8
4.8 US-09a: Aggiunta di una traccia durante la riproduzione *(Assegnato a: Errico Gioia)* . 8

1 Introduzione

Questa sezione documenta la pianificazione del secondo sprint. Lo Sprint 2 costruisce sopra le entità di dominio e i service introdotti nello Sprint 1, aggiungendo la simulazione della riproduzione, l'aggiornamento della UI in tempo reale e l'annullamento delle operazioni (undo).

1.1 Obiettivo dello Sprint

Realizzare la *riproduzione simulata* di tracce e playlist con le tre modalità intercambiabili (sequenziale, shuffle, loop), l'aggiornamento dell'interfaccia in tempo reale (pannello "Now Playing" ed evidenziazione della traccia corrente) e la modifica delle playlist durante la riproduzione. Si aggiunge inoltre la funzionalità di *undo*, predisposta architetturalmente già nello Sprint 1 tramite l'infrastruttura `Command`. Al termine dello Sprint l'utente deve poter avviare, mettere in pausa e saltare la riproduzione di una playlist, cambiarne la modalità a runtime, aggiungere e rimuovere tracce mentre la riproduzione è in corso, vedere lo stato di riproduzione aggiornato in tempo reale e annullare l'ultima operazione di aggiunta o rimozione.

1.2 Stima della velocity

Lo Sprint 1 si è chiuso con una **velocity misurata di 12 Story Points** (12 SP pianificati, 12 completati). Su questa base, per lo Sprint 2 sono stati selezionati **24 Story Points**: tutte le user stories ad Alta priorità ancora aperte (US-07a, US-07b, US-08, US-09a, US-09b, US-10, US-11, per complessivi 19 SP) più una user story a Media priorità, la US-12 *(undo)*, da 5 SP.

Nota di pianificazione. I 24 SP selezionati corrispondono al *doppio* della velocity misurata nello Sprint 1. La scelta è deliberata e motivata da due fattori: (i) le user stories di riproduzione e UI real-time sono fortemente coese e condividono la stessa infrastruttura (`PlaybackService`, `PlaybackStrategy`, property osservabili), per cui affrontarle nello stesso sprint riduce il costo di integrazione; (ii) l'infrastruttura comune già esistente dallo Sprint 1 (Model, Service, `CommandHistory`) abbatte il lavoro di setup rispetto al primo sprint. Per gestire il rischio, le due user stories a priorità più bassa nel piano **US-11** ed **US-12** sono marcate come *buffer*: in caso di scostamento a metà sprint verranno rinviate allo Sprint 3 senza compromettere il nucleo della riproduzione. Coerentemente con il principio di *time-boxing* adottato nel progetto, il completamento integrale dei 24 SP non è un vincolo: si privilegia la qualità del design e del codice rispetto al numero di funzionalità.
La capacity nominale dello Sprint resta di **36 ore-uomo** (9 ore-uomo settimanali per ciascuno dei 4 membri). La velocity reale dello Sprint 2 verrà misurata in sede di Sprint Review e confrontata con la stima per affinare la pianificazione dello Sprint 3.

1.3 User Stories selezionate

Sono state scelte sette user stories di Alta priorità tutte quelle ancora aperte dopo lo Sprint 1 più la prima user story a Media priorità (US-12). Le US-07a, US-07b e US-08 introducono i controlli di riproduzione e il cambio di modalità (pattern Strategy); le US-09a e US-09b consentono la modifica della playlist durante la riproduzione; le US-10 e US-11 realizzano l'aggiornamento della UI in tempo reale (pattern Observer su property JavaFX); la US-12 abilita l'undo completando i metodi `undo()` dei comandi e collegando la `CommandHistory` all'interfaccia.

#	User Story	SP
US-07a	Come utente voglio avviare e mettere in pausa la riproduzione di una traccia o playlist, così da controllare l'ascolto.	3
US-07b	Come utente voglio saltare alla traccia successiva durante la riproduzione, così da scorrere la playlist.	2
US-08	Come utente voglio scegliere la modalità di riproduzione (sequenziale, shuffle, loop), così da personalizzare l'ascolto.	3
US-09a	Come utente voglio aggiungere una traccia a una playlist mentre è in riproduzione, senza interrompere l'ascolto.	2
US-09b	Come utente voglio rimuovere una traccia da una playlist mentre è in riproduzione, senza interrompere l'ascolto.	3
US-10	Come utente voglio vedere la traccia in riproduzione (titolo, autore, tempo) aggiornata in tempo reale.	3
US-11	Come utente voglio vedere la playlist aggiornarsi (traccia corrente evidenziata) mentre la riproduzione procede.	3
US-12	Come utente voglio annullare (undo) una qualsiasi operazione di aggiunta o rimozione.	5
	Totale Story Points	**24**

Table 1: User Stories selezionate per lo Sprint 2.

2 Task Breakdown e assegnazioni

L'organizzazione del lavoro conferma l'approccio a *ownership per user story* adottato nello Sprint 1: ciascuna US viene presa in carico da un singolo membro che la porta dall'inizio alla fine attraversando tutti i layer architetturali (Model, Service, Controller, View, Test). Le user stories sono state raggruppate per affinità tecnica, così che ogni membro lavori su un'area coerente del prodotto.
Come nello Sprint 1, alcune componenti di infrastruttura comuni (lo scheletro del `PlaybackService`, l'interfaccia `PlaybackStrategy` con le tre implementazioni, la `PlaybackStrategyFactory`, l'enum `PlaybackMode` e il meccanismo di simulazione basato su `Timeline`) devono essere definite collettivamente prima che ciascun membro inizi il lavoro autonomo sulla propria US, poiché tutte le user stories di riproduzione vi si appoggiano.

2.1 Legenda e scala di stima

Legenda: FA = Francesco Apicella LC = Lorenzo Campagna RC = Raffaele Cutolo EG = Errico Gioia.
Le ore di sviluppo sono state stimate con la stessa scala di riferimento interna dello Sprint 1: **1 SP = 2.5h**, **2 SP = 4h**, **3 SP = 6h**, **5 SP = 9h**. La progressione non è perfettamente lineare per riflettere la maggiore incertezza delle US più ampie. Coerentemente con la nota di pianificazione, il carico orario complessivo di questo sprint eccede la capacity nominale: l'eccedenza è concentrata sulle US-buffer (US-11 ed US-12), le prime candidate al rinvio in caso di scostamento.

2.2 US 0 – Setup condiviso del motore di riproduzione (10 ore-uomo totali)

Lavoro preparatorio distribuito tra i membri prima dello split sulle singole US:

- Scheletro del `PlaybackService` con le firme dei metodi pubblici (`play`, `pause`, `skip`, `setMode`) e le property osservabili `currentTrack`, `elapsed`, `state` (RC, 1h);
- Definizione dell'interfaccia `PlaybackStrategy` con il metodo `next(Playlist, Track)` e dell'enum `PlaybackMode` (`SEQUENTIAL`, `SHUFFLE`, `LOOP`) (FA, 1h);

- Implementazione delle tre strategie concrete `SequentialStrategy`, `ShuffleStrategy`, `LoopStrategy` e della `PlaybackStrategyFactory` (FA + LC, 2h ore-uomo);
- Setup del meccanismo di simulazione basato su `Timeline` JavaFX (LC, 1h);
- Scheletro del controller `NowPlayingCtrl` e del file `NowPlaying.fxml` (RC, 1h);
- Sessione di design collettiva: contratti fra `PlaybackService` e strategie, esposizione delle property per il binding, gestione della fine-playlist in modalità sequenziale (tutti, 1h ciascuno = 4h ore-uomo).

2.3 Lorenzo Campagna (LC): US-07a + US-07b (5 SP)

Setup condiviso (2h).

- Implementazione delle tre strategie concrete e della factory, in coppia con FA (1h);
- Setup del meccanismo di simulazione `Timeline` (1h).

US-07a – Avvio e pausa della riproduzione (3 SP, 6h).

- `PlaybackService.play(Playlist)` e `pause()`: avvio dalla prima traccia, transizione delle property `state` (`PLAYING`/`PAUSED`) e gestione della ripresa dal punto di pausa conservando `elapsed` (2.5h);
- Pulsanti Play/Pausa nella UI e relativo controller, con binding allo stato del service (2h);
- Test JUnit su `play`/`pause` (avvio, pausa con conservazione del tempo, ripresa) (1.5h).

US-07b – Skip della riproduzione (2 SP, 4h).

- `PlaybackService.skip()` che interroga la `PlaybackStrategy` corrente per la traccia successiva e gestisce il caso di fine playlist in modalità sequenziale (stato `STOPPED`) (1.5h);
- Pulsante Skip nella UI e collegamento al service (1h);
- Test JUnit su `skip` (traccia successiva disponibile; ultima traccia in sequenziale → STOPPED) (1.5h).

Rifinitura e code review (1h).

- Javadoc su `PlaybackService.play`, `pause`, `skip` + test sul codice US-07a e US-07b (0.5h);
- Code review di US-08 e US-09b (FA) (0.5h).

2.4 Francesco Apicella (FA): US-08 + US-09b (6 SP)

Setup condiviso (2h).

- Definizione dell'interfaccia `PlaybackStrategy` e dell'enum `PlaybackMode` (1h);
- Implementazione delle tre strategie concrete e della factory, in coppia con LC (1h).

US-08 – Scelta della modalità di riproduzione (3 SP, 6h).

- `PlaybackService.setMode(PlaybackMode)` che richiede alla `PlaybackStrategyFactory` la strategia corrispondente e la sostituisce a runtime senza interrompere la traccia corrente (2h);
- ComboBox/ToggleGroup di selezione modalità nella UI, applicabile prima o durante la riproduzione (2h);
- Test JUnit sulla factory (`create` per ciascun `PlaybackMode`) e su `setMode` (cambio a runtime, continuità della traccia corrente) (2h).

US-09b – Rimozione di una traccia durante la riproduzione (3 SP, 6h).

- Logica nel `PlaybackService` per la rimozione di una traccia dalla playlist in riproduzione: se la traccia rimossa è diversa da quella corrente l'ascolto prosegue; se coincide con la corrente, si interrompe la simulazione e si forza l'avanzamento (`skip`) secondo la strategia (3h);
- Collegamento del pulsante Remove al flusso di riproduzione (riuso di `RemoveTrackFromPlaylistCommand`) (1h);
- Test JUnit per entrambi gli scenari (rimozione di traccia non corrente; rimozione del brano in esecuzione con avanzamento automatico) (2h).

Rifinitura e code review (1h).

- Javadoc su `setMode`, `PlaybackStrategyFactory` e sulla logica di rimozione live + test sul codice US-08 e US-09b (0.5h);
- Code review di US-07a e US-07b (LC) (0.5h).

2.5 Raffaele Cutolo (RC): US-10 + US-11 (6 SP)

Setup condiviso (2h).

- Scheletro del `PlaybackService` con firme dei metodi pubblici e property osservabili (1h);
- Scheletro di `NowPlayingCtrl` e di `NowPlaying.fxml` (1h).

US-10 – Pannello "Now Playing" in tempo reale (3 SP, 6h).

- Completamento di `NowPlaying.fxml` (label titolo/autore, contatore tempo, `ProgressBar`) (2h);
- Binding in `NowPlayingCtrl` delle property `currentTrack` ed `elapsed` del `PlaybackService` ai componenti grafici, senza callback manuali (2.5h);
- Test JUnit sull'avanzamento delle property del service (aggiornamento di `currentTrack` ed `elapsed` nel tempo) (1.5h).

US-11 – Evidenziazione della traccia corrente nella playlist (3 SP, 6h). ***[US buffer]***

- Rendering personalizzato della riga della `ListView` della playlist che evidenzia la riga corrispondente alla property `currentTrack` (2.5h);
- Aggiornamento automatico dell'evidenziazione a ogni cambio di traccia (manuale o automatico) tramite binding (2h);
- Test JUnit sulla determinazione dell'indice corrente e sul suo aggiornamento al cambio traccia (1.5h).

Rifinitura e code review (1h).

- Javadoc su `NowPlayingCtrl` e sul rendering personalizzato della riga + test sul codice US-10 e US-11 (0.5h);
- Code review di US-12 e US-09a (EG) (0.5h).

2.6 Errico Gioia (EG): US-12 + US-09a (7 SP)

Setup condiviso (1h).

- Partecipazione alla sessione di design collettiva (1h).

US-09a – Aggiunta di una traccia durante la riproduzione (2 SP, 4h).

- Verifica che l'aggiunta di una traccia (tramite `AddTrackToPlaylistCommand`) alla playlist in riproduzione aggiorni la coda senza interrompere l'ascolto, sfruttando il riferimento condiviso alla stessa `Playlist` nel `PlaybackService` (2h);
- Abilitazione del bottone Add anche durante la riproduzione e relativo test (2h).

US-12 – Annullamento di un'operazione, undo (5 SP, 9h). ***[US buffer]***

- Implementazione del metodo `undo()` nelle sei classi `Command` concrete (`AddTrackToLibraryCommand`, `RemoveTrackFromLibraryCommand`, `CreatePlaylistCommand`, `DeletePlaylistCommand`, `AddTrackToPlaylistCommand`, `RemoveTrackFromPlaylistCommand`), ripristinando lo stato precedente tramite i service (4h);
- Collegamento di `CommandHistory.undo()` a un pulsante "Annulla" nella UI, con disabilitazione del pulsante quando lo stack è vuoto (scenario "undo a vuoto") (2.5h);
- Test JUnit sull'`undo` di ciascun comando (ripristino dello stato precedente) e sul comportamento dello stack vuoto (2.5h).

Rifinitura e code review (1h).

- Javadoc sui metodi `undo()` dei comandi e su `CommandHistory.undo()` + test sul codice US-12 e US-09a (0.5h);
- Code review di US-10 e US-11 (RC) (0.5h).

3 Assegnazione delle code review

Si conferma la pratica delle code review incrociate dello Sprint 1, con un nuovo accoppiamento (ruotato rispetto al primo sprint) per allargare la conoscenza incrociata del codice fra i membri del team. Ogni membro rivede il lavoro di un altro:

Owner della US	Reviewer
LC (US-07a, US-07b)	FA
FA (US-08, US-09b)	LC
RC (US-10, US-11)	EG
EG (US-12, US-09a)	RC

Table 2: Assegnazione delle code review fra i membri del team per lo Sprint 2.

4 Dettaglio delle User Stories

4.1 US-07a: Avvio e pausa della riproduzione *(Assegnato a: Lorenzo Campagna)*

- Implementare i metodi `PlaybackService.play(Playlist)` e `pause()`, avviando la riproduzione dalla prima traccia, gestendo la transizione della property `state` e la ripresa dal punto di pausa con conservazione di `elapsed`.
- Realizzare i pulsanti Play/Pausa nell'interfaccia e collegarli al service tramite il controller.

- Scrivere i test JUnit per `play` e `pause` (avvio, pausa con conservazione del tempo trascorso, ripresa dallo stesso punto).
- Redigere la Javadoc per i metodi `play` e `pause` del `PlaybackService`.

4.2 US-07b: Skip della riproduzione *(Assegnato a: Lorenzo Campagna)*

- Implementare `PlaybackService.skip()` delegando alla `PlaybackStrategy` corrente la scelta della traccia successiva e gestendo il caso di fine playlist in modalità sequenziale (passaggio allo stato `STOPPED`).
- Aggiungere il pulsante Skip nell'interfaccia e collegarlo al service.
- Scrivere i test JUnit per `skip` (traccia successiva disponibile; ultima traccia in modalità sequenziale).
- Redigere la Javadoc per il metodo `skip`.

4.3 US-08: Scelta della modalità di riproduzione *(Assegnato a: Francesco Apicella)*

- Implementare `PlaybackService.setMode(PlaybackMode)` che richiede alla `PlaybackStrategyFactory` la strategia corrispondente e la sostituisce a runtime, senza interrompere la traccia in ascolto.
- Realizzare il selettore di modalità (ComboBox o ToggleGroup) nell'interfaccia, utilizzabile prima o durante la riproduzione.
- Scrivere i test JUnit per `PlaybackStrategyFactory.create` (una per ciascun `PlaybackMode`) e per `setMode` (cambio a runtime con continuità della traccia corrente).
- Redigere la Javadoc per `setMode` e per la factory.

4.4 US-09b: Rimozione di una traccia durante la riproduzione *(Assegnato a: Francesco Apicella)*

- Implementare nel `PlaybackService` la logica di rimozione di una traccia dalla playlist in riproduzione: prosecuzione dell'ascolto se la traccia rimossa non è quella corrente; interruzione e avanzamento automatico (`skip`) se coincide con quella in esecuzione.
- Collegare il pulsante Remove al flusso di riproduzione, riusando `RemoveTrackFromPlaylistCommand`.
- Scrivere i test JUnit per entrambi gli scenari (rimozione di traccia diversa dalla corrente; rimozione del brano in esecuzione con avanzamento alla successiva).
- Redigere la Javadoc per la logica di rimozione durante la riproduzione.

4.5 US-10: Pannello "Now Playing" in tempo reale *(Assegnato a: Raffaele Cutolo)*

- Completare il file `NowPlaying.fxml` con le label di titolo e autore, il contatore del tempo trascorso e una `ProgressBar` di avanzamento.
- Implementare in `NowPlayingCtrl` il binding delle property `currentTrack` ed `elapsed` del `PlaybackService` ai componenti grafici, in modo che si aggiornino automaticamente senza callback manuali.

- Scrivere i test JUnit sull'avanzamento delle property del service (aggiornamento di `currentTrack` ed `elapsed` nel tempo).
- Redigere la Javadoc per `NowPlayingCtrl`.

4.6 US-11: Evidenziazione della traccia corrente *(Assegnato a: Raffaele Cutolo)* [US buffer]

- Implementare un rendering personalizzato della riga della `ListView` della playlist che evidenzi visivamente la riga corrispondente alla property `currentTrack`.
- Garantire l'aggiornamento automatico dell'evidenziazione a ogni cambio di traccia (manuale o automatico) tramite il binding alle property del service.
- Redigere la Javadoc per il rendering personalizzato della riga di evidenziazione.

4.7 US-12: Annullamento di un'operazione, undo *(Assegnato a: Errico Gioia)* [US buffer]

- Implementare il metodo `undo()` nelle sei classi `Command` concrete, ripristinando lo stato precedente tramite i rispettivi service (l'infrastruttura era già predisposta nello Sprint 1).
- Collegare `CommandHistory.undo()` a un pulsante "Annulla" nell'interfaccia, disabilitando il pulsante quando lo stack dei comandi è vuoto (scenario "undo a vuoto").
- Scrivere i test JUnit per l'`undo` di ciascun comando (ripristino esatto dello stato precedente) e per il comportamento con stack vuoto.
- Redigere la Javadoc per i metodi `undo()` dei comandi e per `CommandHistory.undo()`.

4.8 US-09a: Aggiunta di una traccia durante la riproduzione *(Assegnato a: Errico Gioia)*

- Verificare e garantire che l'aggiunta di una traccia (tramite `AddTrackToPlaylistCommand`) alla playlist in riproduzione aggiorni la coda in tempo reale senza interrompere l'ascolto, sfruttando il riferimento condiviso alla stessa `Playlist` nel `PlaybackService`.
- Abilitare il bottone Add anche durante la riproduzione.
- Scrivere i test JUnit per l'aggiunta durante la riproduzione (la traccia attiva prosegue senza interruzione, la coda si aggiorna).
- Redigere la Javadoc per il flusso di aggiunta durante la riproduzione.